

# 2D Cricket Game

Zaid Fareed | 23i-0829 | Section – B

GitHub Repository:

[https://github.com/Zaid-fareed/WebProgramming/tree/main/i230289\\_a2](https://github.com/Zaid-fareed/WebProgramming/tree/main/i230289_a2)

## 1. Introduction and Overview

This project is a fully functional 2D Cricket Game built using React and the HTML5 Canvas API. The core objective of the game is to score as many runs as possible within a limit of 12 balls (2 overs) while avoiding losing 2 wickets. Instead of relying on a purely random number generator for the shot outcomes, the game implements a skill-and-luck hybrid mechanic using a dynamically moving "Power Bar" tied to a strict probability mapping system.

## 2. The Probability-Based Power Bar Mechanic

The Power Bar is the central interactive element of the game. It visually represents the probability of various outcomes (Wickets, Dot Balls, and Runs).

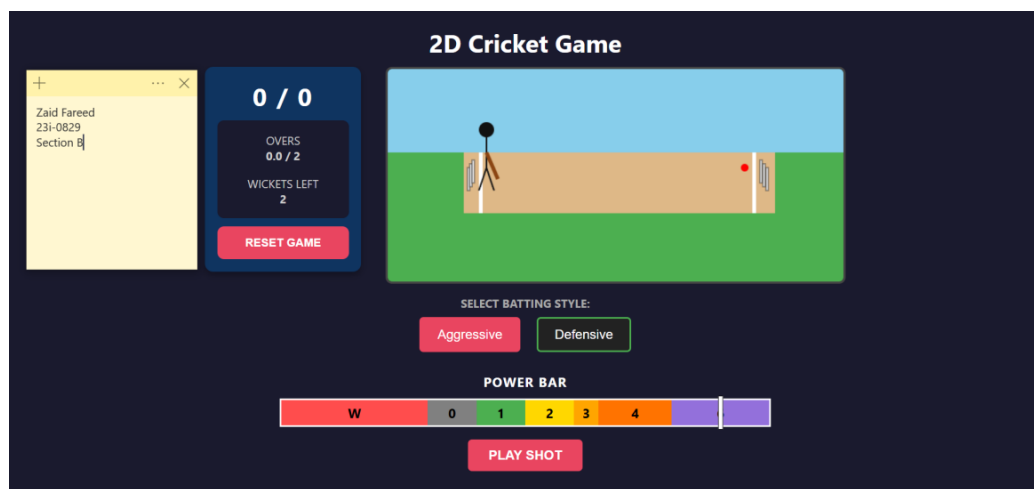
- **Visual Representation:** The bar is divided into colored segments. The width of each segment is dynamically rendered based on its exact probability percentage (width:  $\text{percentage} * 100\%$ ).
- **The Slider:** A slider indicator continuously oscillates left and right across the bar from a value of 0.0 to 1.0 using the browser's `requestAnimationFrame` for smooth performance.
- **Player Interaction:** When the player clicks "PLAY SHOT", the slider's exact position (a float value between 0 and 1) is captured and sent to the game's logic engine to calculate the outcome.

## 3. Batting Styles and Dynamic Probabilities

To introduce strategic decision-making, the game features two distinct batting styles. Toggling between these styles completely changes the underlying probability arrays, forcing the player to adapt their timing.

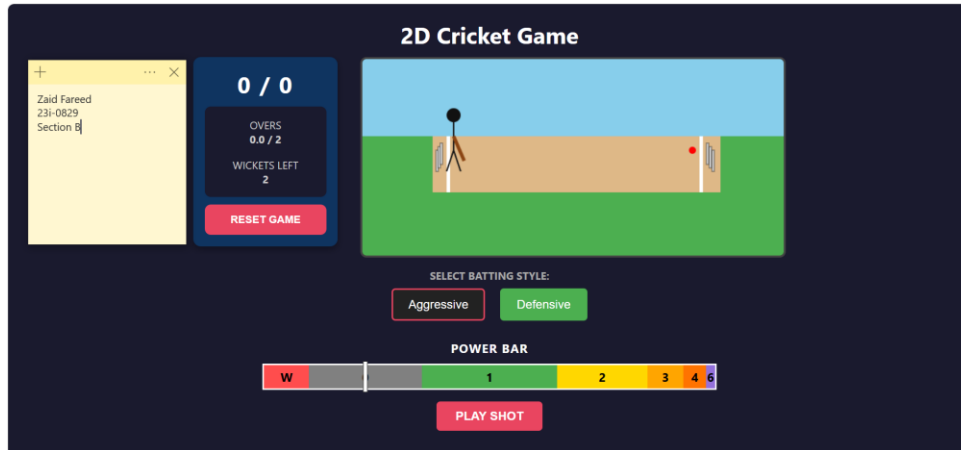
**Aggressive Mode (High Risk, High Reward)** Designed for hitting boundaries, this mode increases the physical width of the "4" and "6" segments on the Power Bar but drastically increases the risk of getting out.

- Wicket: 30%
- 0 Runs (Dot): 10%
- 1 Run: 10%
- 2 Runs: 10%
- 3 Runs: 5%
- 4 Runs: 15%
- 6 Runs: 20%



**Defensive Mode (Low Risk, Low Reward)** Designed for strike rotation and wicket preservation. The chance of getting out drops significantly, but hitting maximum boundaries becomes nearly impossible.

- Wicket: 10%
- 0 Runs (Dot): 25%
- 1 Run: 30%
- 2 Runs: 20%
- 3 Runs: 8%
- 4 Runs: 5%
- 6 Runs: 2%



## 4. Mathematical Evaluation of a Shot

When a shot is played, the game does not just pick a random number. It uses a **Cumulative Probability Algorithm** mapped against the captured position of the Power Bar slider.

1. The system captures the slider's position (e.g.,  $\text{pos} = 0.45$ ).
2. It sets a tracking variable ( $\text{cumulativeSum} = 0$ ).
3. It iterates through the currently active probability array (Aggressive or Defensive).
4. For each possible outcome, it adds that outcome's probability to the  $\text{cumulativeSum}$ .
5. If the  $\text{pos}$  is less than or equal to the  $\text{cumulativeSum}$ , that specific outcome is selected and the loop breaks.

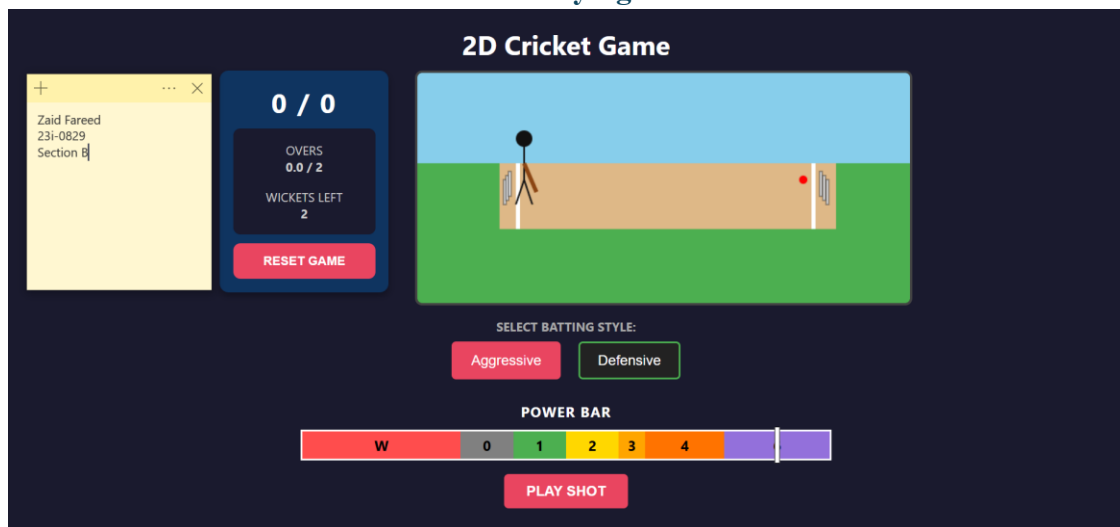
## 5. Game State and Animation Loop Syncing

Once the mathematical outcome is determined, the React application updates its state.

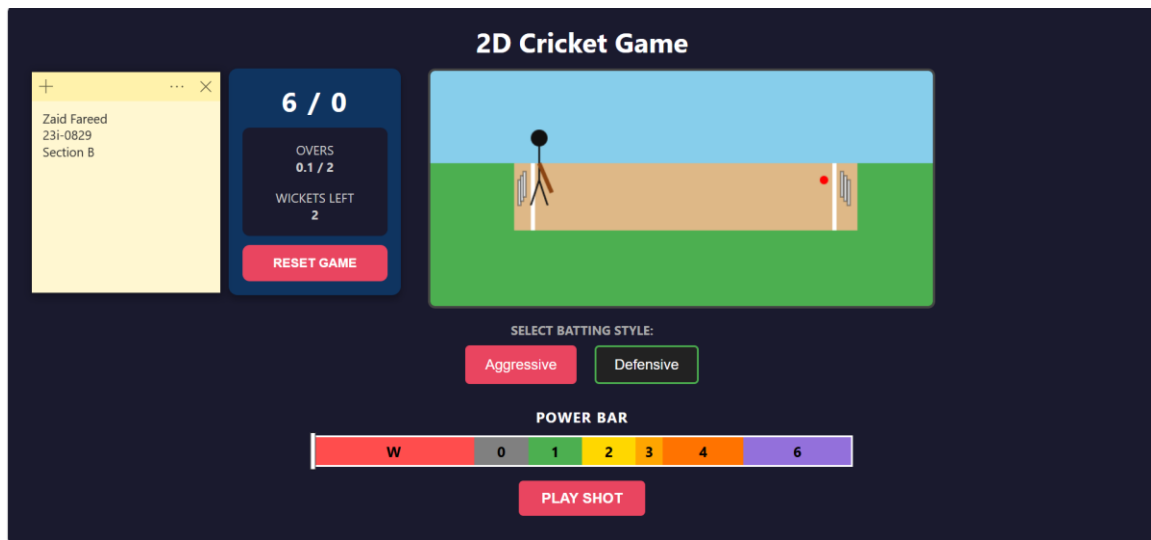
- **Animation Trigger:** The calculated result is passed down to the `Ground.js` Canvas component. This triggers a separate visual animation loop where the ball travels down the pitch, the stickman rotates their bat, and floating text (e.g., "SIX!" or "OUT!") fades into the screen.
- **Scoreboard Update:** To maintain realism, the main React state (runs, wickets, ballsBowled) does not update the scoreboard immediately when the button is clicked. Instead, the `Ground` component utilizes a callback function (`onAnimEnd`) to notify the main App that the visual animation has finished. Only then are the scores officially tallied.

```
const handleAnimationComplete = useCallback(() => {
  if (shotResult) {
    if (shotResult.label === 'W') {
      setWickets((w) => w + 1);
    } else {
      setRuns((r) => r + shotResult.r);
    }
    setBallsBowled((b) => b + 1);
  }
  setIsAnimating(false);
}, [shotResult]);
```

Before Playing shot:



After Playing Shot:



- **Game Over State:** If the wickets reach 2 or the balls reach 12, the application triggers an overlay locking the controls and presenting a final match summary.

